



GDB (The GNU DeBugger)

Amitabha Sanyal
April - 2026



- The GDB model
 - The debugger process and the debugged process
- Basic usage
 - Starting GDB
 - Setting breakpoints
 - Stepping over the program: next, step, finish
 - Examining data
 - Watchpoints
 - Examining the call stack
- Advanced gdb
 - Working with core dumps
 - Writing pretty-printers using the GDB Python API.



Compiling the program for debugging

- Assume that the program being debugged is **buggyprog.cpp**
 - Compile the program as `$ g++ -g buggyprog.cpp -o buggyprog`
 - **-g** provides GDB extra debugging information to connect the executable back to the source program.
 - What are the 10 source statements around the current breakpoint?
 - Which machine instructions correspond to:
`seriesValue += xpow/ComputeFactorial(k);`
 - What is the type of the variable `seriesValue`?
 - What is the memory address where `seriesValue` is stored?
- The **-g** flag adds this information to the executable.
 - `readelf --debug-dump=line` gives line number → machine code map
 - `readelf --debug-dump=info` gives types, location (grep for the variable)

Starting the Debugger



- Start the debugger as:

```
$ gdb buggypr og
```

```
$ gdb --args buggypr og args... (arguments from the command line)
```

```
$ gdb buggypr og coredump (more about this later)
```

- Run the debugger as:

```
$ (gdb) run
```

```
$ (gdb) run arg1 arg2
```

And the program runs to completion (or abortion) with bugs and everything.

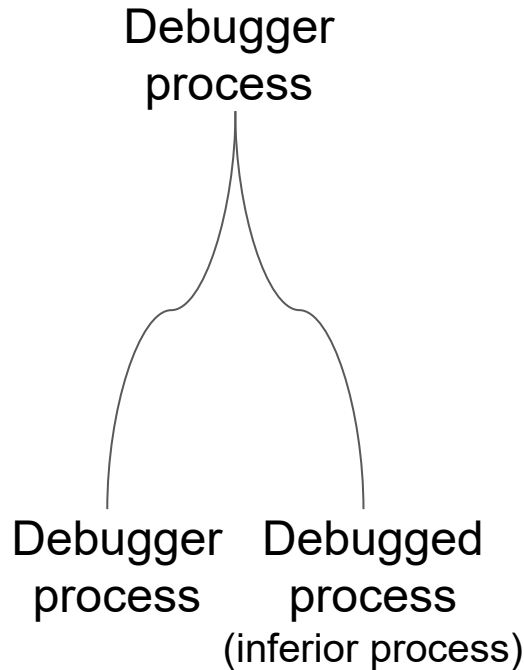
Not very useful.



Debugger Process (gdb)

Reads user input, and executes it:

- Shows info about debugger state - breakpoints, watchpts, convenience vars, macros)
- Shows information about the debugged process (program vars, PC).
- Enables breakpoint setting
- Passes control to debugged process



Debugged Process

- Executes till breakpoint or end.
- Forced to give up control to debugger.



Starting the Debugger: What happens internally

1. **GDB:** user types `break main` and then `run` (alternately `start`)
2. **GDB:** `fork()` → `waitpid()`
3. **Child:** `ptrace(PTRACE_TRACEME)` → `execve("./buggyprog")`
4. **Kernel:** stops child at `execve`, wakes **GDB** via `waitpid()`.
5. **GDB (Wakes):** `set_breakpoint(main)`; hands control back to child
6. **Child (Runs):** Hits `0xCC`; Hardware triggers exception. Call this program point `p`.
7. **Kernel:** Halts child through `SIGTRAP`, wakes **GDB**.
8. **GDB (Wakes):** User debugs and configures additional breakpoints at various program points.
 - a. For each additional program point `p'` do: `set_breakpoint(p')`;
 - b. `execute_instr_at(p)`;
 - c. `ptrace(PTRACE_CONT)`; `waitpid()`

`execute_instr_at(p)`: Restore original instruction at `p`; Execute this instruction; Write `0xCC` back to `p`.

`set_breakpoint(p)`: Read and save original instruction at `p`; Write `0xCC`.

Setting Breakpoints



- Setting breakpoints
 - at line *lineno* of the current file
(gdb) break *lineno*
 - Other variations
(gdb) break *function*
(gdb) break *filename:linenum*
(gdb) break *filename:function*
(gdb) info breakpoints
 - **(gdb) tbreak *lineno/function/filename:linenum/filename:function***

gdb demonstration in accompanying video.

Reaching a breakpoint



- Reaching breakpoints

- To reach the first breakpoint

(gdb) run

Starts the program and stops at the first breakpoint.

- To reach subsequent breakpoints

(gdb) continue

Resumes execution until the next breakpoint is hit.

Stepwise execution



- Stepping over the program line by line
 - **(gdb) next**
executes current line and moves to the next line.
 - **(gdb) step**
executes current line. If the current line is a function, gdb steps into the function.
 - **(gdb) finish**
Continue to the end of the current function

Locating the current point of execution



- Locating the current point
 - **(gdb) where**
Displays the current call stack (i.e., function calls leading to this point).
 - **(gdb) bt**
Equivalent to where; shows the call stack
- Listing statements around the current point
 - **(gdb) list**
Displays source code around the current line (usually 10 lines at a time).



Printing values in gdb

- Printing
 - **print** *exp*
Evaluates and prints any valid C/C++ expression
 - **print** *function::variable*
Prints value of a variable scoped within a function.
 - **print** *file::variable*
Prints value of a variable defined in a specific file.
- Display
 - **display** *exp*
Automatically prints the value of the expression at every step.

Breaking when an expression changes



- Watch expression
 - **watch** *exp*
Pauses execution when the value of the expression changes.
 - **watch** *exp if cond*
Pauses execution only if the value of the expression changes and the condition is true



How to debug a core dump

- Allow core dump creation
 - `ulimit -c unlimited` // `ulimit -c` sets limit on core dump size (default is 0)
 - `sudo sysctl -w kernel.core_pattern=core` // `sysctl -w` writes to the kernel to save the core dump as **core** in **current dir**.
- Compile with **-g**, run and trigger a crash
- Debug the core file using GDB: `gdb ./program core.3669781`

- Inside gdb, you can only *examine* and *not run* the program.
 - **bt** — View call stack (backtrace), **up/down** to navigate the backtrace, **frame *n*** for the *n*th frame
 - **info locals** — Examine variables
 - **disassemble**, **info registers**
 - **info proc mappings**: See the virtual memory map. Did the crash happen because of an access to an unmapped region?



Advanced gdb – The GDB–Python API

- Consider debugging a C++ program with the outline:

```
#include <memory>
#include <string>
```

```
int main() {
    auto node_ptr =
    std::make_unique<Node>(Node{42, "Unique"});
    //Breakpoint here
    ...
}
```

```
(gdb) disable pretty-printer
(gdb) p (node_ptr)
$6 = {
    _M_t = {
        <std::__uniq_ptr_impl<Node,
std::default_delete<Node>>> = {
            _M_t = {
                ...
                <std::_Head_base<1,
std::default_delete<Node>, true>> =
{ _M_head_impl = {<No data fields>}}, <No data
fields>},
            <std::_Head_base<0, Node*, false>> =
{ _M_head_impl = 0x55555556c2b0
```

Advanced gdb – The GDB–Python API



- Consider debugging a C++ program with the outline:

```
struct  node {  
    int  key_value;  
    node *left;  
    node *right;  
}  
  
class  btree {  
public : btree();  
        ~btree();  
        void insert(int key);  
        node *search(int key);  
private : node *root;  
}
```

```
int  main () {  
    int  a [10] = {5, 3, 9, 6, 8,  
                  23, 4, 12, 1, 7};  
  
    btree bt;  
    for (int i=0; i<10; i++)  
        bt.insert(a[i]);  
    return 0;  
}
```

Complete programs available with the accompanying resources.



A normal debugging session

```
$ g++ -g tree.cpp -o tree
```

```
$ gdb tree
```

```
GNU gdb (Ubuntu 8.1.1-0ubuntu1) 8.1.1
```

```
(gdb) b 123
```

```
Breakpoint 1 at 0xf88: file tree.cpp, line 123.
```

```
(gdb) run
```

```
Starting program ... Breakpoint 1, main () at tree.cpp:123
```

```
123         return 0;
```

```
(gdb) p bt
```

```
$1 = {root = 0x555555768e70}
```

```
(gdb) p *(bt.root)
```

```
$4 = {key_value = 5, left = 0x555555768e90, right = 0x555555768eb0}
```


A normal debugging session



- Native GDB output is cumbersome for visualizing complex data (e.g. `std::unique_ptr`, arbitrary user-defined structures).
- The [GDB Python API](#) lets you extend GDB: traverse in-memory objects, synthesize views.
- With a Python script you can:
 - Inspect `bt in tree.cpp` programmatically
 - Emit a **graphviz** dot file representing the tree topology
 - Automatically invoke **graphviz** to render and display the tree within your debug session.

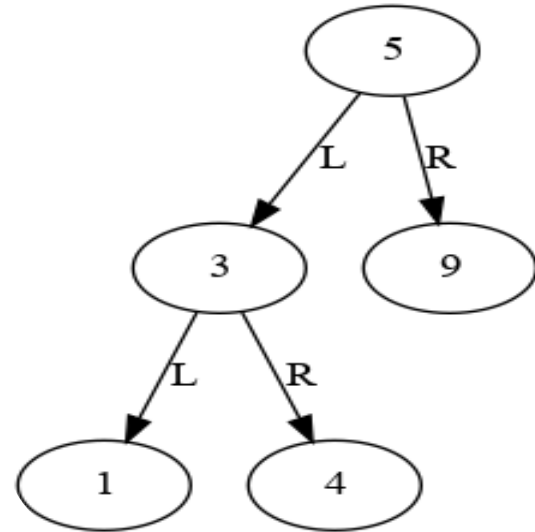
A dot figure



example.dot

```
di graph G {  
  splines=line;  
  node5[ label = "5"];  
  node3[ label = "3"];  
  node1[ label = "1"];  
  node3->node1[ label = "L"];  
  node4[ label = "4"];  
  node3->node4[ label = "R"];  
  node9[ label = "9"];  
  node5->node3[ label = "L"];  
  node5->node9[ label = "R"];  
}
```

xdot example.dot



vscode has extensions for previewing dot files.



Generating the dot file

- To generate the dot file using a python script `test_gdb.py`:

```
import io
def print_tree(nodeptr): # Function to walk and print
                        # the tree

...
outfile = open("out.dot", "w+")
outfile.write("digraph G { \n")
outfile.write("splines=line; \n")
bt_node = gdb.parse_and_eval('bt') # bt_node is a gdb.Value object
print_tree(bt_node['root'])
outfile.write("}")
outfile.close()
```

Generating the dot file



- `gdb.parse_and_eval` takes an C/C++ expression from the debugged program as a string, and parses and evaluates it.
- The result is an object of the type `gdb.Value`.
- [This link](#) describes how values in the debugged program are handled in Python.
- The value of `bt.root` as a python object is passed to `print_tree()`.



print_tree

```
1 def print_tree(nodeptr):
2     if nodeptr == 0x0:
3         return ""
4     else :
5         node = nodeptr.dereference()
6         node_name = "node"+str(node['key_value'])
7         outfile.write(node_name + "[ label = "+ "\n"
8             +str(node['key_value']) + "\n" + "];\n")
9         left_name = print_tree(node['left'])
10        right_name = print_tree(node['right'])
11        if left_name != "":
12            outfile.write(node_name + "-" + left_name
13                + "[ label = \"L\" ]" + ";\n")
14        if right_name != "":
15            outfile.write(node_name + "-" + right_name
16                + "[ label = \"R\" ]" + ";\n")
17        return node_name
```

create a dotfile node

#print the left tree

#print the right tree

#edges from node to subtrees

comments on print_tree



- Line 5: If **val** is a **gdb.Value** object corresponding to a pointer in the debugged language, then **val.dereference()** gives the **gdb.Value** of its pointee.
- Lines 7,9,10: All **structs**, **classes** and **unions** are represented by dictionary **gdb.Value** objects. Their components can be extracted by the dictionary notation.

How to use the Python script?

- After setting a breakpoint, run the debugged program, till you hit the break point. Now say:
(gdb) source test_gdb.py
- The script will be executed and the structure of **bt** will be printed in **out.dot**

Customizing a **gdb** printer for **btree**



- Instead of sourcing the Python script at every breakpoint, embed logic directly into **gdb**'s pretty-printer.
- Define a printer for the **btree** class and register it with the GDB Python API.
- Now, when you run:

```
$ (gdb) p bt
```

gdb will automatically invoke your printer to:

- Traverse the current **bt** structure in memory
- Emit a graphviz dot description of the tree
- The dot viewer renders the new state of **bt**.



Customizing a gdb printer for `btree`

- Define a class called **TreePrinter** which must have
 - A constructor which stores a value of the type **btree**.
 - A function **to_string** which is the string representation of the pretty-printed value. However we write into a dot file and return the empty string instead.

In addition:

- A function, in this case called **lookup_type**, which relates an object of type **btree** to a **TreePrinter** object.
- The pretty printer is registered by adding this function to **`gdb.pretty_printers`**.
- One can use the gdb initialization script **`.gdbinit`** to source **`tree_printer.py`**.



Creating and registering a type based printer.

```
import io

def print_tree(nodeptr):
    ...

class TreePrinter:
    def __init__(self, val):
        self.val = val

    def to_string(self):
        global outfile
        nodeptr = self.val['root']
        outfile = open("out.dot", "w+")
        outfile.write("digraph G { \n")
        outfile.write("splines=line; \n")
        print_tree(nodeptr)
        outfile.write("}")
        outfile.close()
        return ""
```

```
def lookup_type (val):
    if str(val.type) == 'btree':
        return TreePrinter(val)
    return None
gdb.pretty_printers.append
(lookup_type)
```

In summary:

- Write a class **XPrinter** whose constructor has a member of type **X**, and a function **to_string**.
- define a function **lookup_type** that takes a value **val** of type **X** and returns instance **Xprinter(val)**
- register **lookup_type**.
- source file in **~/gdbinit**
- **(gdb) print val:** gdb retrieves the pretty-printer for **val**, does **Xprinter(val).to_string()**

Triggering actions based on events



- The python interface for gdb events are described [here](#).
- Lets say, we want to print a `bt r e e` every time the debugged process stops.
 - This is what `display` does
- We write a callback function, also called a handler (`stop_handler`) for a `stop` event.
- The parameter called `event` contains details of the event. Not useful in the present case.
- `gdb.events.stop.connect (stop_handler)` is the call that connects the event `stop` with the handler `stop_handler`.



Triggering actions based on events

```
import gdb

def stop_handler(event): // this is what happens on "display bt"

    bt_val = gdb.parse_and_eval('bt')

    # Force GDB to run its internal pretty printing lookup loop.
    # This automatically invokes Treeprinter(bt_val).to_string()
    printer_output = str(bt_val)

gdb.events.stop.connect(stop_handler)
```



- **`gdb`** provides fine-grained control over program execution:
 - Breakpoints, stepping, backtraces, variable inspection, and more.
 - Essential for understanding control flow and program state.
- The GDB-Python API extends this with programmability:
 - Automate actions (e.g., on stop, on breakpoint hit)
 - Build custom pretty-printers for complex data structures
 - Trigger external tools — like visualizing trees with `graphviz`

Conclusion



- [*GDB: The GNU Project Debugger*](#): The bible
- [*GDB man pages*](#): For quick reference
- [*Debugging with GDB: The GNU Source-Level Debugger*](#): Richard Stallman, Roland Pesch, et al. 12th Media Services, 2018.
 - All of gdb in a single book written by its original creator.
- [*YouTube: GDB Tutorials by Jacob Sorber*](#)
 - A series of videos on different aspects of gdb
- [*GDB Python API Documentation*](#)
 - For writing pretty-printers, event handlers, etc.